
BANKING MODERN DATA STACK

A production-grade data engineering pipeline
from OLTP to analytics-ready dashboards



PostgreSQL



Kafka



Snowflake



dbt



Airflow



Power BI

WHAT I BUILT

An end-to-end real-time banking data pipeline



6-Table OLTP Schema

Customers, accounts, transactions, loans, branches, and audit log with ENUMs and triggers



Real-Time CDC

Debezium captures every INSERT, UPDATE, DELETE from PostgreSQL WAL in real time



Medallion Architecture

Bronze (raw) → Silver (typed) → Gold (marts) with SCD Type-2 history tracking via dbt



Enterprise Dashboards

Power BI connected to Snowflake Gold layer with 4 dashboard pages

THE PIPELINE

How data flows from source to dashboard



THE HARD PROBLEM

Debezium doesn't send clean data

Debezium's JsonConverter base64-encodes PostgreSQL NUMERIC types, sends dates as days-since-epoch, and timestamps as microsecond integers. Most tutorials skip this entirely.

WHAT DEBEZIUM SENDS

```
"annual_income": "IOl6"  
"date_of_birth": 13429  
"created_at": 1707003388657140
```

WHAT IT ACTUALLY MEANS

```
annual_income: $21,569.22  
date_of_birth: 2006-10-04  
created_at: 2024-02-04 03:36
```

**Built a custom Snowflake JavaScript UDF to decode
base64 byte arrays back to usable dollar amounts.**

THE OPTIMIZATION

From hours to minutes

V1

~11 HOURS

One INSERT per record

New connection per row

3 seconds per record

Open → Insert → Commit → Close
... repeated 40,000 times

V2

~3 MINUTES

One INSERT per file

One connection per table

3 seconds per 100 records

LATERAL
FLATTEN(PARSE_JSON(...))
... batch insert in one query

220x faster

Pipelines break between systems, not inside them.

DATA MODELING WITH DBT

From raw CDC payloads to business-ready marts

BRONZE

5 tables

Raw Debezium CDC payloads stored
as Snowflake VARIANT (semi-structured JSON)

SILVER

4 staging views + 2 SCD
snapshots

Typed, deduplicated, Debezium-decoded.
Custom UDF for base64 NUMERIC,
epoch-to-timestamp conversions

GOLD

2 dimensions + 3 fact tables

dim_customers, dim_accounts,
fct_transactions, fct_daily_balances,
fct_loan_performance

BY THE NUMBERS

10

Docker
Services

6

OLTP
Tables

40K+

CDC
Events

11

dbt
Models

8

Phases
Built

220x

Speed
Improvement

LINEAGE GRAPH + DASHBOARDS



Snowflake interface showing a SQL query and its results.

Workspaces: Databases
My Workspace
+ Add new
Banking DW Setup.sql
Test.sql

SQL Query:

```
GROUP BY a.customer_id
ORDER BY account_count DESC;

SELECT COUNT(*) FROM BANKING_DW.SILVER_GOLD.DIM_CUSTOMERS;
SELECT COUNT(*) FROM BANKING_DW.SILVER_GOLD.DIM_ACCOUNTS;
SELECT COUNT(*) FROM BANKING_DW.SILVER_GOLD.FCT_TRANSACTIONS;
SELECT COUNT(*) FROM BANKING_DW.SILVER_GOLD.FCT_LOAN_PERFORMANCE;

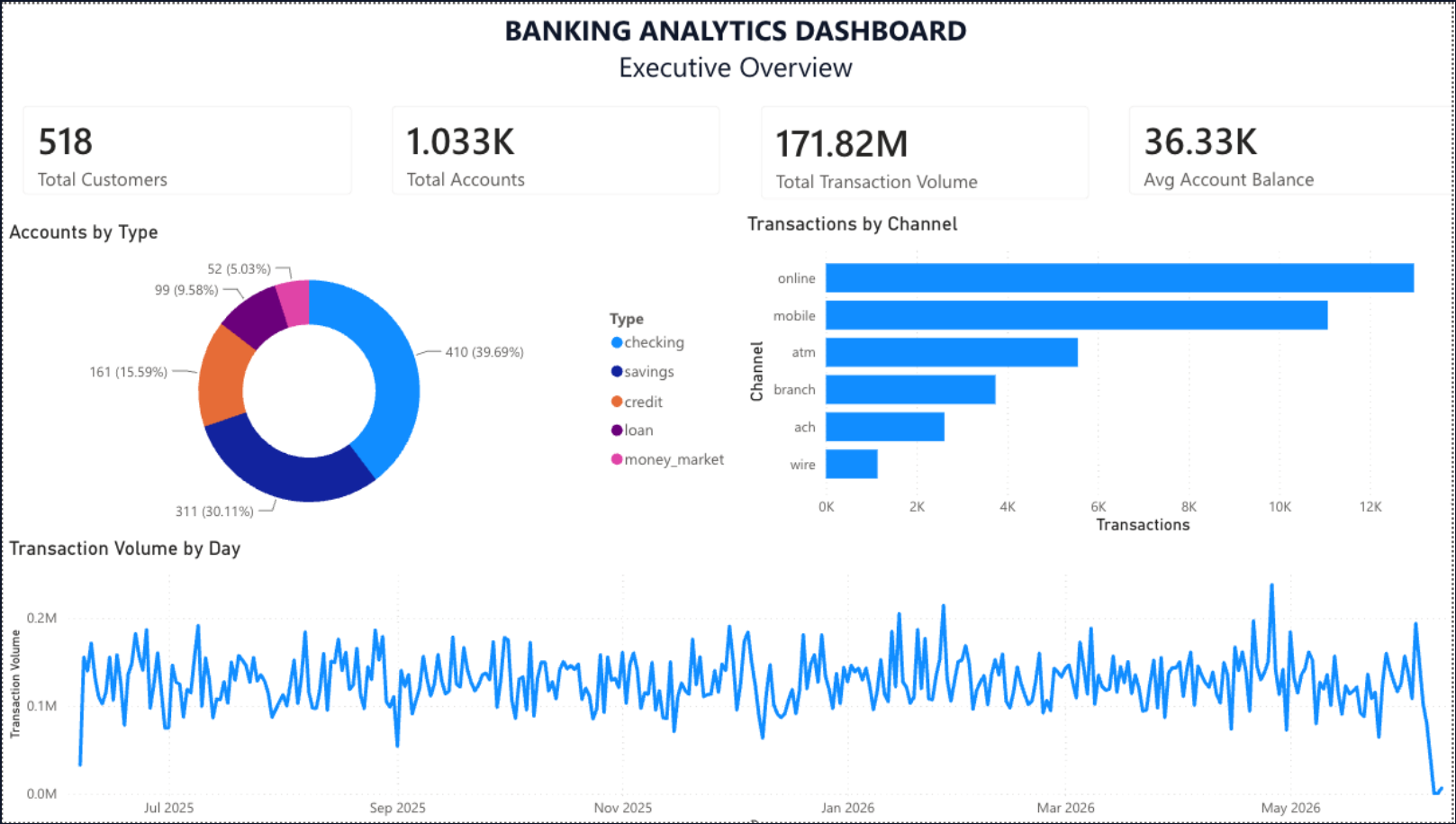
SELECT
  customer_id,
  full_name,
  credit_tier,
  customer_segment,
  total_balance,
  total_loans
FROM BANKING_DW.SILVER_GOLD.DIM_CUSTOMERS
LIMIT 10;
```

Results (just now)

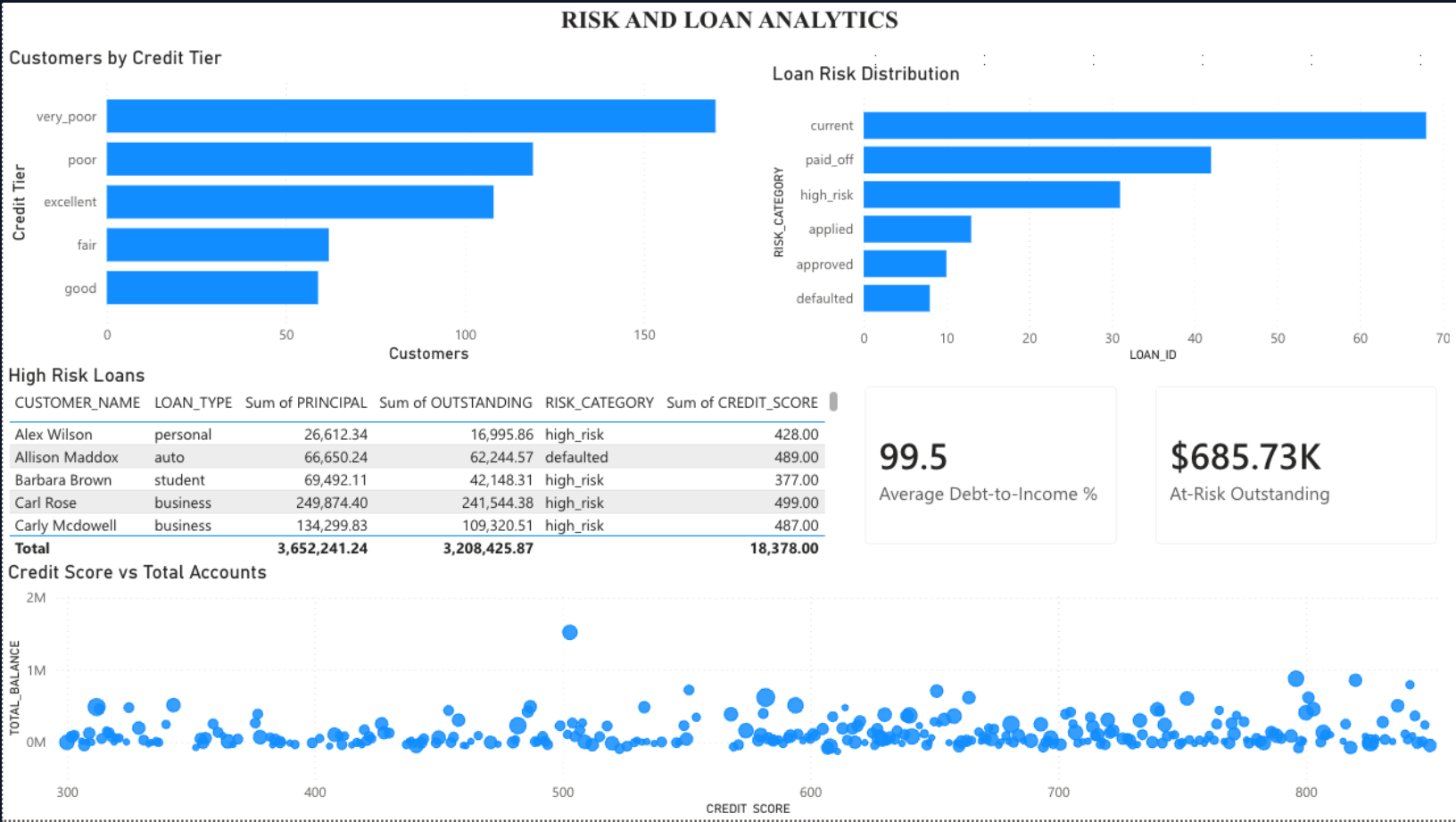
	CUSTOMER_ID	FULL_NAME	CREDIT_TIER	CUSTOMER_SEGMENT	TOTAL_BALANCE	TOTAL_LOANS
	11db3156-8f79-48f5-b3fa-a5851ab5b...	Daniel Johnson	poor	standard	-24543.22	0
	34e1f32e-84c4-42b5-9bb1-d63a5758c...	Diane Johnson	excellent	mid_value	181114.48	1
	+8 more	+8 more	+2 more	+1 more		
1	e34723a2-e63e-458a-b833-abacd4dd114f	Daniel Johnson	poor	mid_value	87091.44	0
2	7cfb1ccf-81c2-4773-8afa-b3ac40c48e1c	Sara Allison	very_poor	standard	11978.67	0
3	77761b14-93d8-4f78-8bd6-60d8623e9891	Gary Hill	poor	standard	9325.08	0
4	b4e6d58b-6938-4af5-ac58-219276b1ea15	Katherine Camacho	excellent	standard	-24543.22	0
5	34e1f32e-84c4-42b5-9bb1-d63a5758c02c	Sherry Williams	poor	standard	-17358.26	0
6	11db3156-8f79-48f5-b3fa-a5851ab5b0aa	Rachel Gray	poor	mid_value	61830.37	0
7	549a24fe-3dde-46cd-89f6-cf191a3f1263	Sandy Johnson	excellent	standard	-20860.12	0
8	f676f78c-6c0c-4d4e-a500-43d7a933cd07	Richard Howard	fair	high_value	181114.48	1

Feedback

DASHBOARDS



DASHBOARDS



CI/CD WITH GITHUB ACTIONS

Every push triggers automated validation

CI — Pull Requests

- ✓ dbt compile
- ✓ dbt run (CI schema)
- ✓ dbt test
- ✓ Isolated CI_TEST schema

CD — Merge to Main

- ✓ dbt snapshot (SCD-2)
- ✓ dbt run (production)
- ✓ dbt test (validation)
- ✓ Path-filtered triggers

The screenshot shows the GitHub Actions interface for a workflow named 'CI - Lint and Test' in the repository 'omkarb40 / Banking-data-pipeline'. The workflow run 'CI - Lint and Test #1' is shown as successful, triggered manually 2 minutes ago, with a status of 'Success' and a total duration of '1m 37s'. The workflow file 'ci.yml' is shown, triggered on 'workflow_dispatch'. A job 'dbt Compile and Test' is shown with a duration of '1m 25s'. The 'Annotations' section shows 1 error and 1 warning. The error message is: 'dbt Compile and Test: Process completed with exit code 1.' The warning message is: 'dbt Compile and Test: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5. Actions will be forc... Show more'.

omkarb40 / Banking-data-pipeline

Q. Type [/](#) to search

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

← CI - Lint and Test

✓ CI - Lint and Test #1

Re-run all jobs

Summary

All jobs

dbt Compile and Test

Run details

Usage

Workflow file

Manually triggered 2 minutes ago

omkarb40 → 2161862 main

Status

Success

Total duration

1m 37s

Artifacts

-

ci.yml

on: workflow_dispatch

dbt Compile and Test 1m 25s

Annotations

1 error and 1 warning

dbt Compile and Test

Process completed with exit code 1.

dbt Compile and Test

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5. Actions will be forc...

Show more

The full repo is public on GitHub.



github.com/omkarb40/Banking-data-pipeline

If you're building a data engineering portfolio
or preparing for interviews, feel free to fork it,
break it and learn from the mistakes I made.

Omkar Badadale

MS Data Science

#DataEngineering #ApacheKafka #Snowflake
#dbt #Airflow #ModernDataStack #CDC